A Project Proposal Report on

SastoBazar Nepal: A Console-Based Online Shopping System

Submitted in Partial Fulfilment of the Requirements for

The Course **Object-Oriented Programming (C++)**

Under Westcliff University

Submitted by:

Sujal Shrestha, WU257800

Date:
02/08/2026

Department of Computer Science / Information Technology

King's College

ABSTRACT

"SastoBazar Nepal" is a console-based (command-line) shopping program written in C++. It lets a customer browse products from five categories, such as electronics, traditional wear, food, handicrafts, and trekking gear, search for items, add them to a cart, and check out using a choice of shipping method and a choice of Nepali payment method, including Cash on Delivery, eSewa, Khalti, IME Pay, ConnectIPS, and Bank Transfer. A separate admin panel lets a shop administrator add, update, and remove products, and view the list of registered customers.

Shopping by visiting stores in person takes time, and it does not scale well across a country with as much geographic spread as Nepal, where delivery time and cost change a lot between provinces. This project models that reality directly: shipping fees and delivery times are calculated per province, across all seven provinces of Nepal, and value-added tax (VAT) is added automatically to every order.

This project is also built to clearly demonstrate the four main ideas of object-oriented programming. Abstraction is shown through abstract classes and interfaces (Entity, IDisplayable, BaseUser, Payment, ShippingStrategy). Encapsulation is shown through private and protected fields with controlled getters and setters. Inheritance is shown through Customer and Admin extending BaseUser, Product extending Entity, and six payment classes extending Payment. Polymorphism is shown through virtual functions and two factory methods, createPayment() and createShipping(), which return a base-class pointer but run the correct subclass's code at runtime.

Keywords: Product, Customer, Admin, Payment, Shipping, Polymorphism, C++

TABLE OF CONTENTS

LIST OF FIGURES

1. INTRODUCTION

This project is an online shopping program that runs in a command-line window (console). A customer can register, log in, browse products by category, search by keyword, add items to a cart, choose a shipping method, choose a payment method, and check out. A shop administrator can log in separately to manage the product catalog. This section explains the problem the project tries to solve, its goals, and what it does and does not cover.

1.1 PROBLEM STATEMENT

Shopping in person takes time and travel, and it becomes harder the further a customer lives from a city center. In a country like Nepal, delivery cost and delivery time can vary a great deal between provinces. A shopping system that ignores this, and simply charges one flat delivery fee for everyone, does not reflect how delivery actually works.

At the same time, Nepali customers do not all pay the same way. Some prefer Cash on Delivery, while others prefer a digital wallet such as eSewa, Khalti, or IME Pay, a payment switch such as ConnectIPS, or a direct bank transfer. A shopping system that only supports one payment method does not fit how people in Nepal actually pay for things.

1.2 PROJECT OBJECTIVES

The main goals of this project are:

1. Build a working shopping program that runs from the command line, using core C++ and object-oriented programming ideas: abstraction, encapsulation, inheritance, and polymorphism.

2. Let a customer register, log in, browse products by category, search by keyword, and view full product details.
3. Let a customer add products to a cart, remove items, and see a running cart total before checking out.
4. Let a customer choose a shipping method and a payment method at checkout, with the shipping fee and delivery estimate calculated based on the customer's province, and value-added tax (VAT) added automatically.
5. Let a customer view past orders and track an order's delivery status.
6. Let a shop administrator log in separately to add, update, or remove products, and to view the list of registered customers.

1.3 SCOPE AND LIMITATIONS

In this project, a program is built where a customer can register, browse a catalog of sample Nepali products, and place an order with a chosen shipping method and payment method, while an administrator manages that catalog, all inside one running console window.

The scope of this project includes:

- A menu-driven console program written in C++, built around abstract classes and interfaces (Entity, IDisplayable, BaseUser, Payment, ShippingStrategy) and their concrete subclasses.
- Customer features: registration, login, browsing, searching, cart management, checkout, order history, order tracking, and account updates.

- Six payment methods (Cash on Delivery, eSewa, Khalti, IME Pay, ConnectIPS, Bank Transfer) and two shipping methods (Standard, Express), each calculated across Nepal's seven provinces.
- An admin panel for managing the product catalog (add, update price, update stock, remove) and viewing registered customers.

The limitations of this project are:

- The program stores all data (products, customers, and orders) in memory only. Once the program is closed, this data is lost, since there is no file or database storage yet.
- There is no real connection to eSewa, Khalti, IME Pay, ConnectIPS, or any bank. Each payment method simulates the steps a customer would go through (such as entering a mobile number or OTP), but no real transaction takes place.
- The program is text-based and runs in a terminal window; it does not have a graphical user interface (GUI).
- There is only one admin account, and its username and access code are fixed in the program itself rather than managed through a setup screen.

2. LITERATURE REVIEW

Before building this project, it helps to look at how online shopping and digital payment are normally handled in Nepal, and how similar beginner projects are usually built.

Nepal already has established online shopping platforms and digital wallets. Daraz and SastoDeal are widely used for browsing and ordering a wide range of products, while eSewa, Khalti, IME Pay, and ConnectIPS are commonly used for digital payment, alongside direct bank transfer and cash on delivery. Delivery services in Nepal also generally charge more, and take longer, for provinces that are further from the Kathmandu valley, such as Karnali and Sudurpashchim. This project reflects that same real-world pattern: shipping fee and delivery time both depend on the customer's province.

Many beginner and student projects that teach object-oriented programming use a similar idea: a console-based shopping or billing system built with classes for products, customers, and orders, with a menu that loops until the user chooses to exit. What sets this project apart from a very basic version of that idea is its use of abstract classes and interfaces, and its use of two factory methods, createPayment() and createShipping(), which let new payment or shipping options be added later by writing one new class, without changing the checkout logic itself. This project follows that same beginner approach for its console menu, while going further with its class design to demonstrate abstraction and polymorphism clearly.

3. METHODOLOGY

The following approach will be used to build the project step by step, and to keep the work organized and on track.

3.1 TOOLS TO BE USED

The following tools will be used to build and manage this project.

VISUAL STUDIO CODE:

Visual Studio Code (VS Code) will be used as the main code editor to write, run, and debug the C++ program.

GITHUB:

GitHub will be used to store the project's source code, track changes made over time, and back up the work as it progresses.

3.2 TECHNOLOGIES TO BE USED

This project uses a single programming language and its standard library.

C++:

C++ is the programming language used to build the entire application. Object-oriented programming features of C++ are used throughout: abstract classes and pure virtual functions (Entity, IDisplayable, IManageable, BaseUser, Payment, ShippingStrategy) provide abstraction; private and protected member variables with public getters and setters provide encapsulation; class hierarchies such as BaseUser to Customer/Admin, and Payment to its six subclasses, provide inheritance; and virtual functions called through base-class pointers, returned by the

createPayment() and createShipping() factory methods, provide polymorphism. The C++ Standard Template Library (STL) is used for storing lists of data, mainly through vectors and strings, and standard library headers such as iostream (for input and output), iomanip (for formatting tables and prices), and ctime (for recording the order date) are used throughout the program.

4. SYSTEM DESIGN

This section shows how the program is structured, using a use case diagram, two class diagrams, and a sequence diagram. These diagrams show what the program does, how the classes and interfaces relate to each other, and how a typical checkout is processed.

4.1 USE CASE DIAGRAM

A use case diagram shows what each user can do in the program. In this project, there are two actors: the Customer and the Admin. The diagram below shows the actions each actor can take. Checkout includes two smaller steps, choosing a shipping method and choosing a payment method, shown as included use cases.

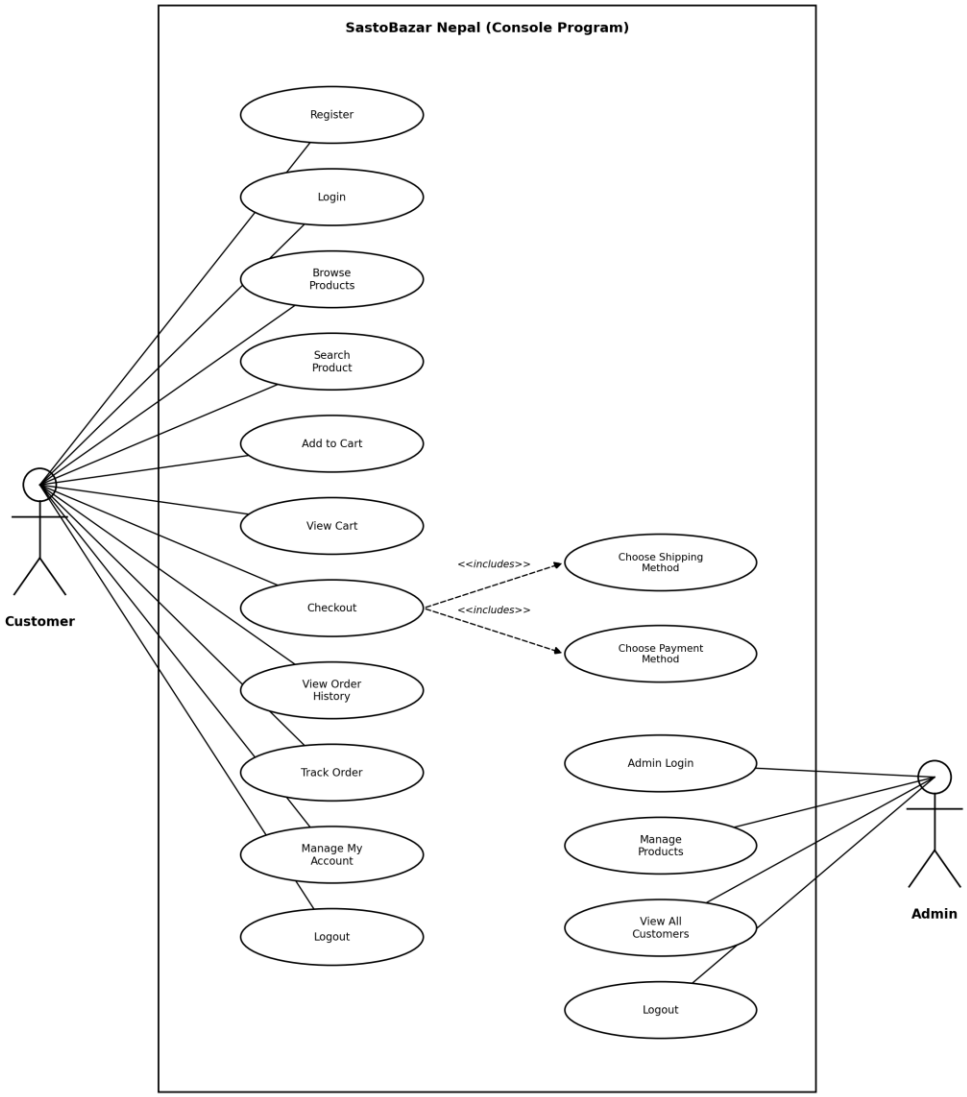

Figure 2: Use Case Diagram

4.2 CLASS DIAGRAM

The class design is shown across two diagrams, split by topic so that each stays readable. The first diagram shows the core entities and user hierarchy. The second diagram shows the Payment and ShippingStrategy hierarchies, which are the clearest examples of polymorphism in this project.

Entity is an abstract base class that gives every object an id and a name. Product inherits from Entity and implements the IDisplayable interface, which requires a display() and a displayDetail() method. BaseUser is an abstract class that gives every user an email, password, phone, address, and province, and requires each subclass to implement its own showMenu() and displayInfo(). Customer and Admin both inherit from BaseUser: Customer adds a shopping cart and order history, while Admin adds an admin access code. ShopSystem holds the overall list of products and customers, plus the single Admin account.

Figure 3: Class Diagram — Core Entities and Users

Payment is an abstract class with six subclasses: CashOnDelivery, ESewa, Khalti, IMEPay, ConnectIPS, and BankTransfer. Each one implements processPayment() in its own way, for example, ESewa asks for a mobile number and OTP, while BankTransfer asks for a bank name and account number. ShippingStrategy is a similar abstract class with two subclasses, StandardShipping and ExpressShipping, each calculating its own fee and delivery estimate per province. ShopSystem never needs to know which exact subclass it is using: its createPayment() and createShipping() factory methods return a Payment or ShippingStrategy pointer, and the correct subclass code runs automatically. This is the polymorphism this project is built to demonstrate.

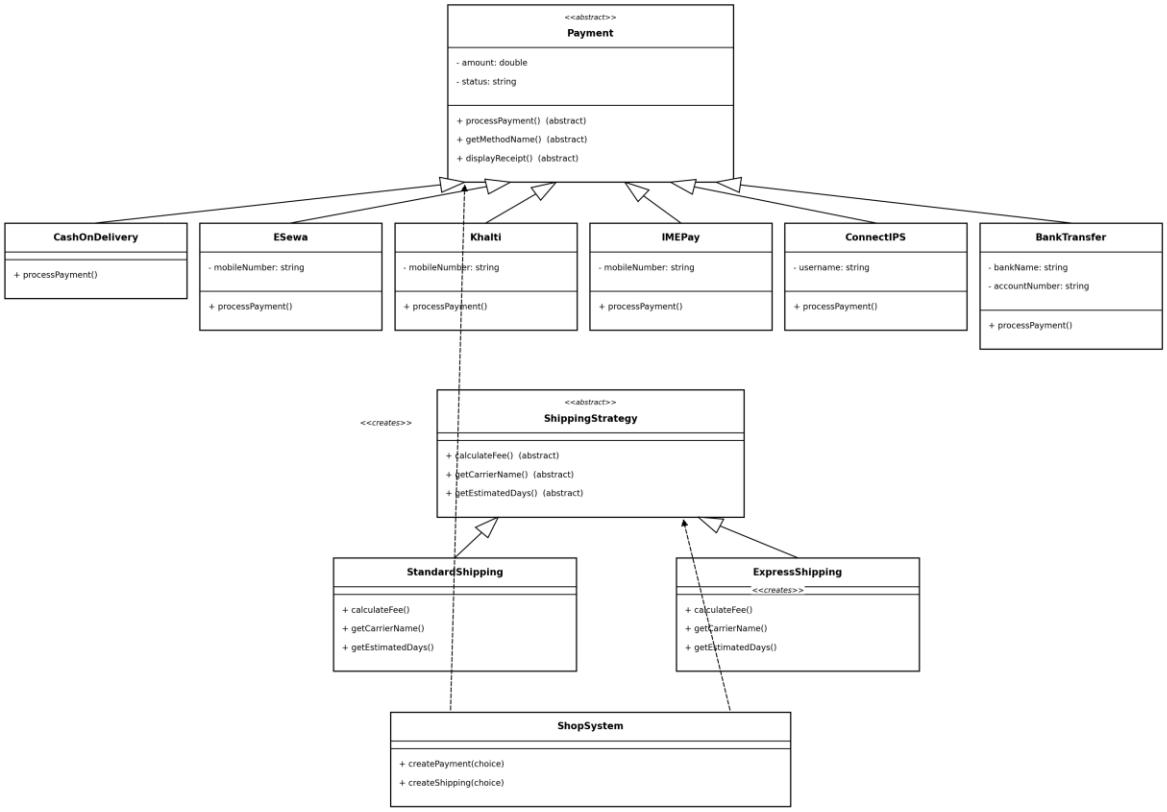

Figure 4: Class Diagram — Payment and Shipping Strategy (Polymorphism)

4.3 SEQUENCE DIAGRAM

A sequence diagram shows the order in which messages are exchanged between objects for one specific task. The diagram below shows the checkout process, from the moment the customer chooses to check out, through choosing a shipping method and a payment method (both resolved polymorphically through the factory methods), to the point where the order is saved and a confirmation and receipt are shown, matching the checkout() function in the program.

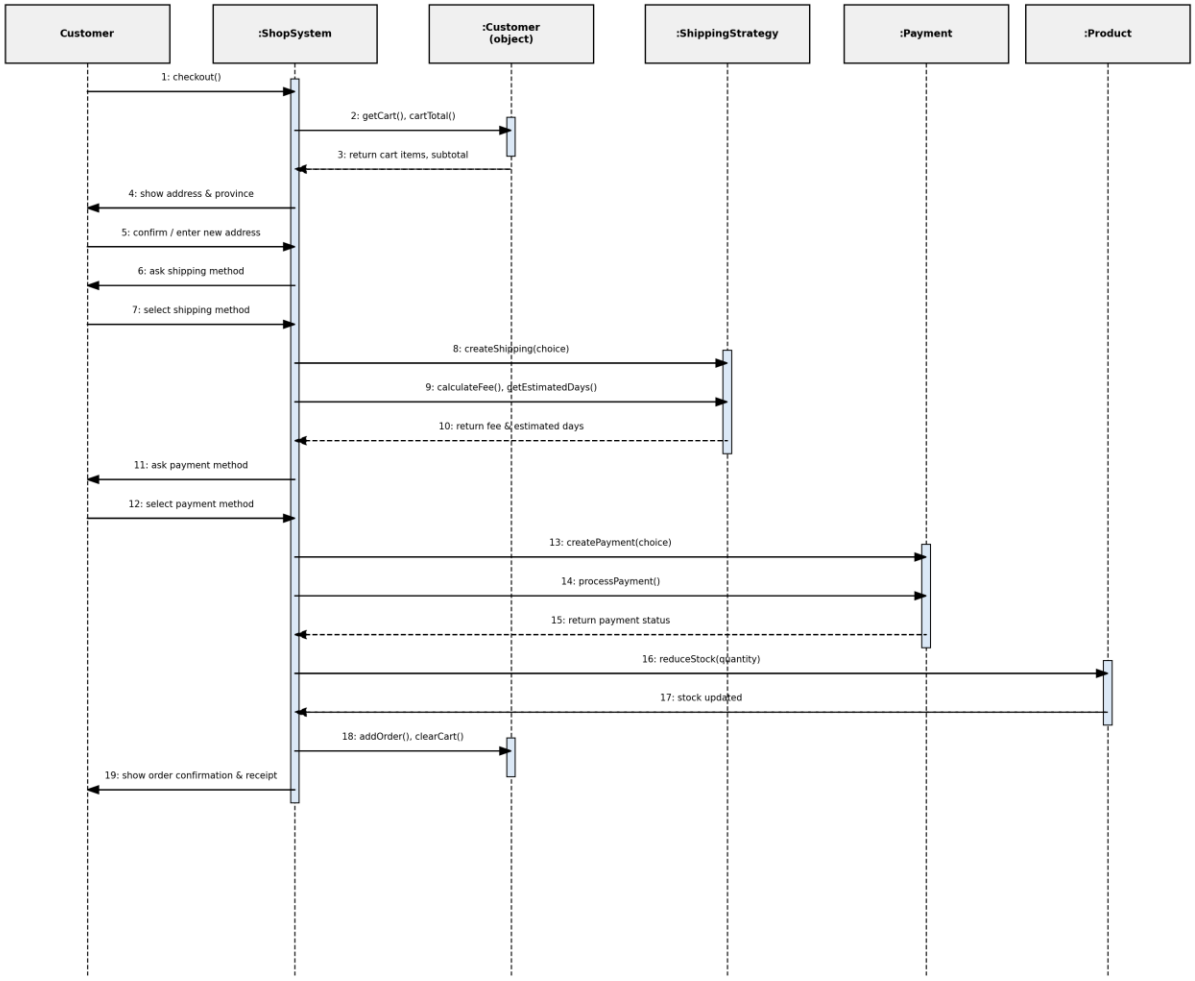

Figure 5: Sequence Diagram — Checkout

5. PROPOSED DELIVERABLES

The final deliverable is a single, working C++ console program named SastoBazar Nepal. It can be compiled and run from a terminal window on any computer with a C++ compiler installed.

Once running, the program will let the customer:

- Register a new account (choosing a province out of all seven) and log in with a username and password.
- Browse all products, browse by category (Electronics, Traditional Wear, Food, Handicrafts, Trekking), or search by keyword.
- View full details of a single product, including price, stock, and description.
- Add products to a cart, remove items, and see the running total of the cart.
- Check out by choosing or updating a delivery address and province, choosing Standard or Express shipping, choosing one of six payment methods, and confirming the order, with VAT and shipping fee added automatically.
- View past orders, see full order details, and track an order's delivery status.
- Update basic account details such as email, phone, address, and province.

The program will also let the shop administrator:

- Log in separately using an admin username, password, and admin access code.
- View all products currently in the catalog.
- Add a new product, update a product's price or stock, or remove a product entirely.
- View the list of all registered customers, along with their email and province.

Since the program keeps its data in memory only, the sample product catalog will reload automatically each time the program starts, and any accounts or orders made during a session will be cleared when the program is closed.

7. BIBLIOGRAPHY

- Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.
- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
- Roger S. Pressman, Ph.D., Software Engineering: A Practitioner's Approach, 7th Edition, McGraw Hill, 2010.
- https://cplusplus.com/reference/
- https://en.cppreference.com/w/
- https://code.visualstudio.com/docs
- https://docs.github.com/